数组与广义表

阅读顺序

点标题跳到对应页

数组的顺序存储 第 11 页
从各维长度推导地址偏移。

特殊矩阵压缩 第 17 页
对称矩阵先确定存哪半边，再计算下标。

稀疏矩阵三元组 第 31 页
转置时先数各列的非零元，再计算起始位置。

行逻辑联接顺序表 第 44 页
用 rpos 定位每行起点，注意最后一行的终点。

十字链表插入 第 58 页
把同一个非零元同时接入有序的行链和列链。

广义表 第 60 页
分别计算最外层长度和嵌套深度。

易错点

二维数组按行优先存放时，跨一行要跳过该行的列数。

第 53 页若未给 rpos 增设哨兵，最后一行的扫描终点应取非零元总数加 1。

随堂自测

广义表 (a,(b,c)) 的长度和深度各是多少？

答案 长度为 2，深度为 2。

第五章　数组

5.1 数组的类型定义

前几章讨论的线性结构中的数据元素都是非结构的原子类型，元素的值是不再分解的，本章讨论的两种数据结构---数组和广义表可以看成是线性表在下述含义的扩展：表中的数据元素本身也是一个数据结构。

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 稀疏矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的表示方法



5.1 数组的类型定义

ADT Array {

数据对象：

$$D = \{a_{j_1,j_2,\ldots,j_i,j_n} \mid j_i = 0,\ldots,b_i - 1,\ i=1,2,..,n\ \}$$

数据关系：

$$R = \{R1, R2, \ldots, Rn\}$$

$$Ri = \{<a_{j_1,\ldots j_i,\ldots j_n}, a_{j_1,\ldots j_i+1,\ldots j_n}> \mid 0 \le j_k \le b_k - 1,$$

$$1 \le k \le n \text{ 且 } k \ne i,\ 0 \le j_i \le b_i - 2,\ i=2,\ldots,n\ \}$$

基本操作：

} ADT Array

二维数组的定义：

数据对象：

$$D = \{a_{ij} \mid 0 \le i \le b_1-1,\ 0 \le j \le b_2-1\}$$

数据关系：

$$R = \{ ROW, COL \}$$

$$ROW = \{<a_{i,j}, a_{i+1,j}> \mid 0 \le i \le b_1-2,\ 0 \le j \le b_2-1\}$$

$$COL = \{<a_{i,j}, a_{i,j+1}> \mid 0 \le i \le b_1-1,\ 0 \le j \le b_2-2\}$$



基本操作：

InitArray(&A, n, bound1, ..., boundn)

DestroyArray(&A)

Value(A, &e, index1, ..., indexn)

Assign(&A, e, index1, ..., indexn)



InitArray(&A, n, bound1, ..., boundn)

操作结果： 若维数 n 和各维长度合法，则构造相应的数组A，并返回OK。



DestroyArray(&A)

操作结果：**销毁数组A。**



Value(A, &e, index1, ..., indexn)

初始条件： A是n维数组，e为元素变量，随后是n 个下标值。

操作结果： 若各下标不超界，则e赋值为所指定的A 的元素值，并返回OK。



Assign(&A, e, index1, ..., indexn)

初始条件： A是n维数组，e为元素变量，随后是n 个下标值。

操作结果： 若下标不超界，则将e的值赋给所指定的A的元素，并返回OK。



5.2 数组的顺序表示和实现

类型特点:

1) 只有引用型操作，没有加工型操作;

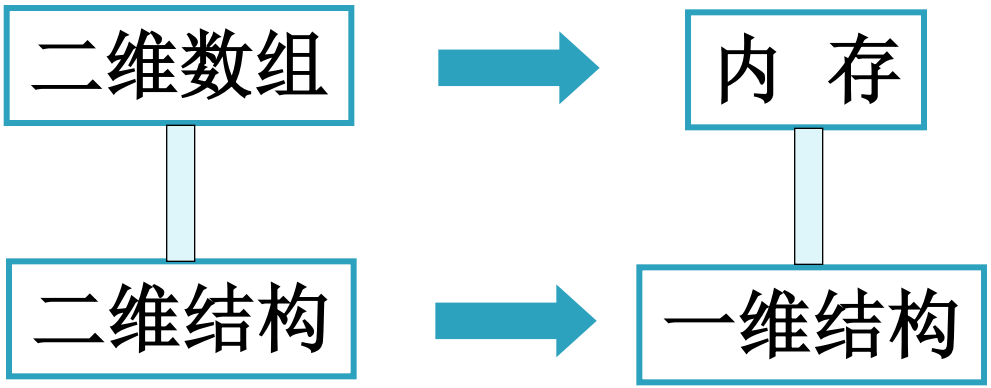
2) 数组是多维的结构，而存储空间是一个一维的结构。

有两种顺序映象的方式:

1)以行序为主序(低下标优先);

2)以列序为主序(高下标优先)。

数组的存储结构与寻址——二维数组



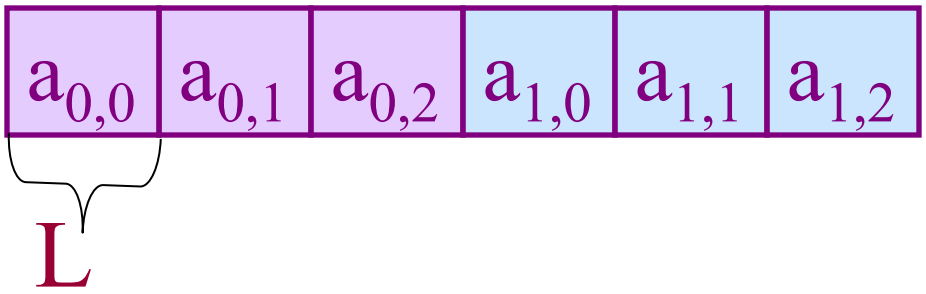
常用的映射方法有两种：

- **按行优先：先行后列，先存储行号较小的元素，行号相同者先存储列号较小的元素。**
- **按列优先：先列后行，先存储列号较小的元素，列号相同者先存储行号较小的元素。**

以“行序为主序”的存储映象

例如：

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$



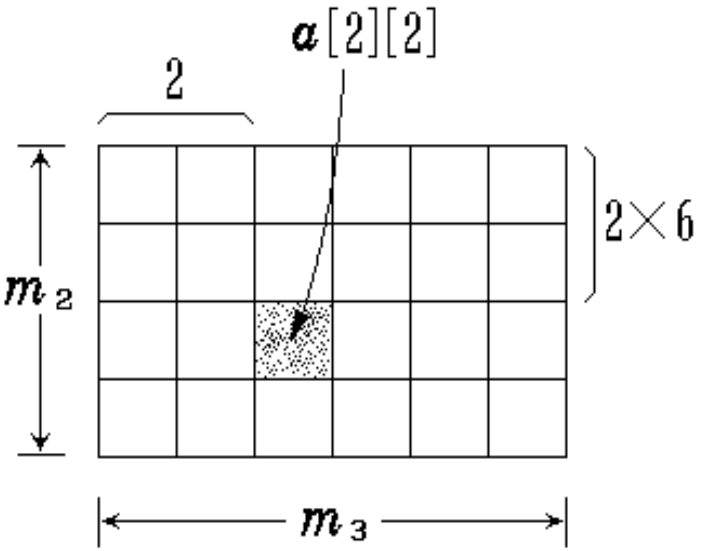
二维数组A中任一元素$a_{i,j}$ 的存储位置

$$LOC(i,j) = LOC(0,0) + (b_2 \times i + j) \times L$$

称为**基地址**或基址。

二维数组

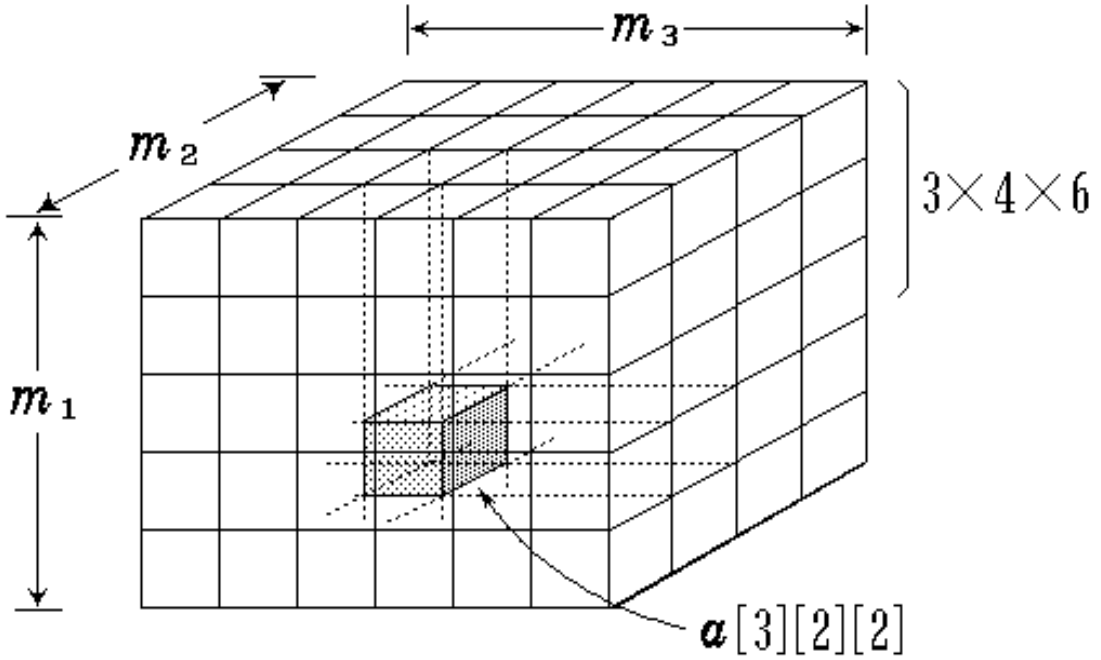
$m_1=5$　$m_2=4$　$m_3=6$



行向量　下标 i

列向量　下标 j

三维数组



页向量　下标 i

行向量　下标 j

列向量　下标 k

三维数组

- 各维元素个数为 m_1, m_2, m_3
- 下标为 i_1, i_2, i_3的数组元素的存储地址：

(按页 / 行 / 列存放)

$$LOC(i_1, i_2, i_3) = a + (\underbrace{i_1 * m_2 * m_3}_{\text{前}i_1\text{页总元素个数}} + \underbrace{i_2 * m_3}_{\text{第}i_1\text{页前}i_2\text{行总元素个数}} + \underbrace{i_3}_{\text{第}i_2\text{行前}i_3\text{列元素个数}}) * l$$

推广到一般情况，可得到 n 维数组数据元素存储位置的映象关系:

$$LOC(j_1, j_2, \ldots, j_n) = LOC(0,0,\ldots,0) + (b_2 \times .. \times b_n \times j_1 + b_3 \times .. \times b_n \times j_2 + b_n \times j_{n-1} + j_n) \times L$$

$$LOC(j_1, j_2, \ldots, j_n) = LOC(0,0,\ldots,0) + \sum_{i=1}^{n} c_i j_i$$

其中 $c_n = L$，$c_{i-1} = b_i \times c_i$，$1 < i \leq n$。

称为 n 维数组的映象函数。**数组元素的存储位置是其下标的线性函数。**

5.3 矩阵的压缩存储

矩阵中非零元素呈某种规律分布或者矩阵中出现大量的零元素的情况下，为了节省存储空间，需要对它们进行“**压缩存储**”，即为多个值相同的元素分配一个存储空间；对零元不分配空间。

值相同的元素或者零元素在矩阵中的分布是否有规律：**特殊矩阵**和**稀疏矩阵**。

特殊矩阵

如果矩阵中值相同的元素或零元素在矩阵中的分布有一定规律，则称之谓"**特殊矩阵**"。

常见的特殊矩阵：对角矩阵、上（下）三角矩阵、对称矩阵、……

存储映象方法

由于特殊矩阵中的非零值元素在矩阵中的分布有着一定的规律，则可将这些非零值元连续存储在一个一维数组中。即矩阵中的任何一个元素和它们在一维数组中的位置之间存在着某种"转换关系"，并且这种转换关系可以用数学公式表示之。。

对称矩阵的压缩存储

- **设有一个 n×n 的对称矩阵 A。**

$$\mathrm{A}=\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

- **对称矩阵中的元素关于主对角线对称，**

  $a_{ij}=a_{ji},\quad 0\le i,j\le n-1$

- 为节约存储，只存对角线及对角线以上的元素，或者只存对角线或对角线以下的元素。前者称为上三角矩阵，后者称为下三角矩阵。

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

下三角矩阵

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$

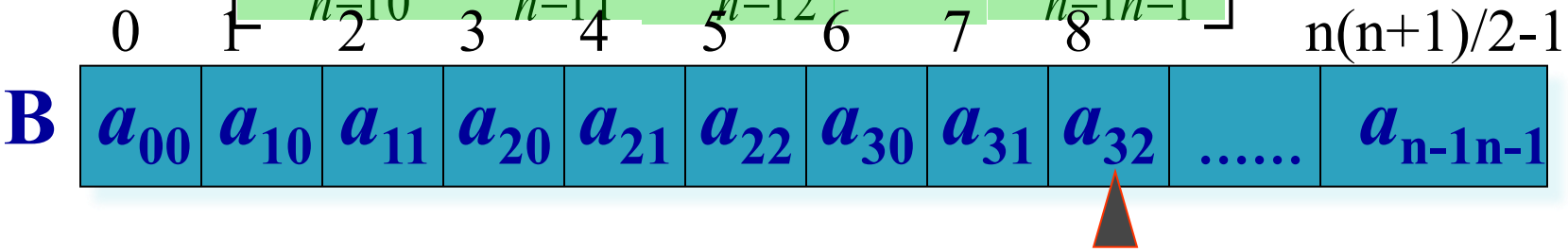
上三角矩阵

- 把它们按行存放于一个一维数组 B 中，称之为对称矩阵 A 的压缩存储方式。
- 数组 B 共有 n + (n – 1) + … + 1 =

  n*(n+1) / 2 个元素。

下三角矩阵

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & \cdots & a_{0n-1} \\ a_{10} & a_{11} & a_{12} & \cdots & a_{1n-1} \\ a_{20} & a_{21} & a_{22} & \cdots & a_{2n-1} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{n-10} & a_{n-11} & a_{n-12} & \cdots & a_{n-1n-1} \end{bmatrix}$$



若 $i \geq j$, 数组元素A[i][j]在数组B中的存放位置为

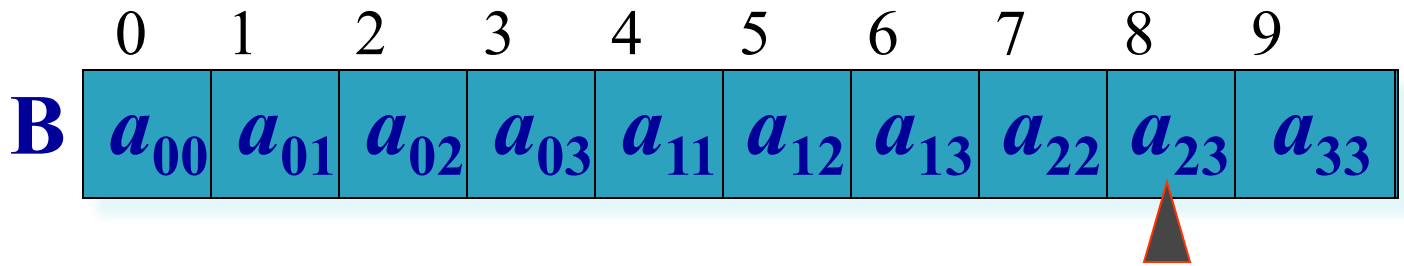
$$1 + 2 + \cdots + i + j = (i + 1) * i / 2 + j$$

前i行元素总数　第i行第j个元素前元素个数

- 若 $i < j$，数组元素 A[i][j] 在矩阵的上三角部分，在数组 B 中没有存放，可以找它的对称元素A[j][i]：　$j*(j+1) / 2 + i$

$$n=4\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix}$$

上三角矩阵



若$i \le j$，数组元素A[i][j]在数组B中的存放位置为

$$n + (n\text{-}1) + (n\text{-}2) + \cdots + (n\text{-}i\text{+}1) + j\text{-}i$$

前i行元素总数　第i行第j个元素前元素个数

- 若 $i \le j$，数组元素A[i][j]在数组B中的存放位置为

$$n + (n-1) + (n-2) + \cdots + (n-i+1) + j-i =$$

$$= (2*n-i+1) * i / 2 + j-i =$$

$$= (2*n-i-1) * i / 2 + j$$

- 若$i > j$，数组元素A[i][j]在矩阵的下三角部分，在数组 B 中没有存放。因此，找它的对称元素A[j][i]。A[j][i]在数组 B 的第 $(2*n-j-1) * j / 2 + i$ 的位置中找到。

何谓稀疏矩阵?

假设 m 行 n 列的矩阵含 t 个非零元素，则称

$$\delta = \frac{t}{m \times n}$$

为**稀疏因子**。

通常认为 $\delta \leq 0.05$ 的矩阵为稀疏矩阵。

以常规方法，即以二维数组表示高阶的稀疏矩阵时产生的**问题：**

1) 浪费空间；

2)是指在进行矩阵的运算中进行了很多与零元素的运算 - 浪费时间。

解决问题的目标:

1) 尽可能少存或不存零值元素;

2) 尽可能减少没有实际意义的运算;

3) 操作方便。 即:

能尽可能快地找到与下标值(i，j)对应的元素，

能尽可能快地找到同一行或同一列的非零值元。

- 但由于非零元素的分布一般是没有规律的，因此在存储非零元素的同时，还必须同时记下它所在的行和列的位置（i,j)。反之，一个三元组(i,j,a_{ij})唯一确定了矩阵A的一个非零元。因此，稀疏矩阵可由表示非零元的三元组及其行列数唯一确定。

$$\begin{vmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{vmatrix}$$

((1,2,12) , (1,3,9) , (3,1,-3) , (3,6,14) ,

(4,3,24) , (5,2,18) , (6,1,15) , (6,4,-7))

<6,7>

稀疏矩阵的压缩存储方法:

一、三元组顺序表

二、行逻辑联接的顺序表

三、十字链表

一、三元组顺序表

```
#define  MAXSIZE  12500
typedef struct {
    int  i, j;      //该非零元的行下标和列下标
    ElemType  e;    // 该非零元的值
} Triple;  // 三元组类型
typedef union {
 Triple  data[MAXSIZE + 1]; //data[0]未用
  int     mu, nu, tu;
} TSMatrix;  // 稀疏矩阵类型
```

Data域中三元组行序为主序

$$\begin{bmatrix} 0 & 0 & 9 & 0 & -7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 16 & 0 \end{bmatrix}$$

TSMatrix a

a.mu=5, a.nu=6,a.tu=6

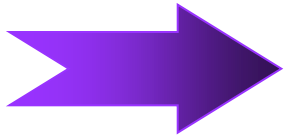
i	j	V
1	3	9
1	5	-7
3	4	8
4	1	5
4	6	2
5	5	16

a.data

如何求转置矩阵?

$$\begin{bmatrix} 0 & 0 & 9 & 0 & -7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & 16 & 0 \end{bmatrix}$$

TSMatrix M



$$\begin{bmatrix} 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 0 \\ -7 & 0 & 0 & 0 & 16 \\ 0 & 0 & 0 & 2 & 0 \end{bmatrix}$$

TSMatrix T

I	J	v
1	3	9
1	5	-7
3	4	8
4	1	5
4	6	2
5	5	16

M.data

I	J	v
3	1	9
5	1	-7
4	3	8
1	4	5
6	4	2
5	5	16

I	J	v
1	4	5
3	1	9
4	3	8
5	1	-7
5	5	16
6	4	2

T.data

```
Status TransposeSMatrix(TSMatrix M, TSMatrix &T){
  T.mu = M.nu;  T.nu = M.mu;  T.tu = M.tu;
  if (T.tu) {
   q=1;
 for (col=1; col<=M.nu; ++col) // 原矩阵的列对应转置后的行
       for (p=1; p<=M.tu; ++p)
 if(M.data[p].j==col){
   T.data[q].i = M.data[p].j; T.data[q].j = M.data[p].i;
   T.data[q].e = M.data[p].e; ++q;}
          } // if
  return OK;
} // TransposeSMatrix
```



分析算法TransposeSMatrix的时间复杂度：

```
for (col=1; col<=M.nu; ++col) … …
for (p=1; p<=M.tu; ++p) … …
```

时间复杂度为: O(M.nu*M.tu)

当tu和mu*nu同数量级的时候:
时间复杂度为: $O(M.mu*M.nu^2)$

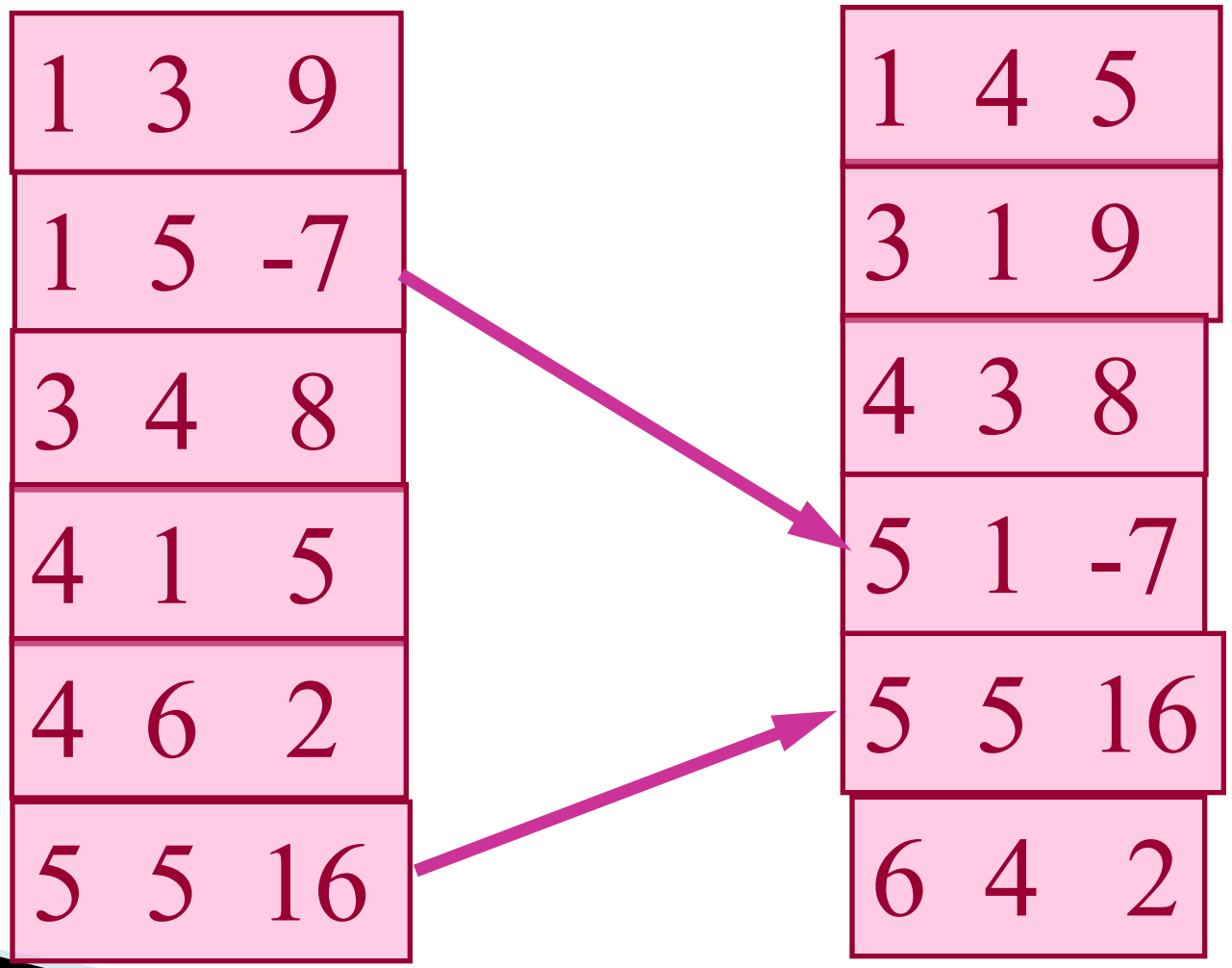


用常规的二维数组表示时的算法

```
for (col=1; col<=nu; ++col)
    for (row=1; row<=mu; ++row)
      T[col][row] = M[row][col];
```

其时间复杂度为: O(mu×nu)

用“三元组”表示时如何实现?



首先应该确定每一行的第一个非零元在三元组中的位置。

1	3	9
1	5	-7
3	4	8
4	1	5
4	6	2
5	5	16

col	1	2	3	4	5	6
Num[pos]	1	0	1	1	2	1
Cpot[col]	**1**	**2**	**2**	**3**	**4**	**6**

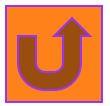
```
cpot[1] = 1;
for (col=2; col<=M.nu; ++col)
    cpot[col] = cpot[col-1] + num[col-1];
```



```
Status FastTransposeSMatrix(TSMatrix M, TSMatrix &T){
  T.mu = M.nu;  T.nu = M.mu;  T.tu = M.tu;
  if (T.tu) {
    for (col=1; col<=M.nu; ++col)  num[col] = 0;
    for (t=1; t<=M.tu; ++t)  ++num[M.data[t].j];
    cpot[1] = 1; // cpot 保存各列在转置表中的起始位置
    for (col=2; col<=M.nu; ++col)
      cpot[col] = cpot[col-1] + num[col-1];
    for (p=1; p<=M.tu; ++p) { 转置矩阵元素 }
  } // if
  return OK;
} // FastTransposeSMatrix
```



```
Col = M.data[p].j;
q = cpot[col]; // 取该列的下一个可用位置
T.data[q].i = M.data[p].j;
T.data[q].j = M.data[p].i;
T.data[q].e = M.data[p].e;
++cpot[col]
```



分析算法FastTransposeSMatrix的时间复杂度：

```
for (col=1; col<=M.nu; ++col) … …
for (t=1; t<=M.tu; ++t) … …
for (col=2; col<=M.nu; ++col) … …
for (p=1; p<=M.tu; ++p) … …
```

时间复杂度为: O(M.nu+M.tu)



二、行逻辑联接的顺序表

三元组顺序表又称**有序的双下标法**，它的特点是，非零元在表中按行序有序存储，因此**便于进行依行顺序处理的矩阵运算**。然而，若需随机存取某一行中的非零元，则需从头开始进行查找。

修改前述的稀疏矩阵的结构定义，增加一个数据成员rpos，其值在稀疏矩阵的初始化函数中确定。

```
#define  MAXMN  500
typedef struct {
    Triple  data[MAXSIZE + 1];
    int     rpos[MAXMN + 1];
    int     mu, nu, tu;
} RLSMatrix;   // 行逻辑链接顺序表类型
```

例如：给定一组下标，求矩阵的元素值

```
ElemType value(RLSMatrix M, int r, int c) {
  p = M.rpos[r]; // 从第 r 行的首个非零元开始
  while (M.data[p].i==r &&M.data[p].j < c)
      p++;
  if (M.data[p].i==r && M.data[p].j==c)
     return M.data[p].e;
  else return 0;
} // value
```



矩阵乘法的精典算法:

```
for (i=1; i<=m1; ++i)
  for (j=1; j<=n2; ++j) {
    Q[i][j] = 0;
    for (k=1; k<=n1; ++k)
        Q[i][j] += M[i][k] * N[k][j];
  }
```

其时间复杂度为: O(m1×n2×n1)

- 1）乘积矩阵Q的非零元仅由M和N中的非零元相乘得到，换句话说，为求得两个稀疏矩阵相乘的乘积，只需要对 M 和 N 中的非零元进行运算即可；
- 2）从矩阵相乘的规则得知，并非两者中每个元素都要进行彼此相乘，**对 M 中的每个非零元，只要在 N 中查找其行号等于它的列号的非零元相乘即可；**
- 3）在上述算法中，Q 的每个元素是由 M 中的一行和N中的一列相应元素连续相乘相加得到的，但在行逻辑链接的三元组顺序表中要连续找到同一列的元素是很不方便的，因此需要改变计算的顺序，寻找新的算法。

结论：

- 要顺着M中的非零元依次处理。
- 对于M中的每个非零元，找到所有N中的行号与其列号相等的元素进行处理。
- 对于Q中的元素是一行一行进行处理的。

两个稀疏矩阵相乘(Q=M×N)的过程大致描述如下：

```
Q初始化;
if  Q是非零矩阵 {  // 逐行求积
   for (arow=1; arow<=M.mu; ++arow) {
                    //  处理M的每一行
     ctemp[] = 0;          // 累加器清零
    计算Q中第arow行的积并存入ctemp[] 中;
    将ctemp[] 中非零元压缩存储到Q.data;
   } // for arow
} // if
```

例：Q = M×N

$$M=\begin{bmatrix} 0 & 1 & 0 \\ 2 & 0 & -4 \\ -1 & 0 & 0 \\ 0 & -2 & 3 \end{bmatrix}$$

$$N=\begin{bmatrix} 2 & 0 & 0 & 6 \\ 0 & -4 & 0 & 0 \\ 0 & -1 & 0 & 3 \end{bmatrix}$$

M.rpos

1	2	3	4
1	2	4	5

N.rpos

1	2	3
1	3	4

$$Q=\begin{bmatrix} 0 & -4 & 0 & 0 \\ 4 & 4 & 0 & 0 \\ -2 & 0 & 0 & -6 \\ 0 & 5 & 0 & 9 \end{bmatrix}$$

```
Status MultSMatrix
    (RLSMatrix M, RLSMatrix N, RLSMatrix &Q) {
 if (M.nu != N.mu) return ERROR; // 左矩阵列数须等于右矩阵行数
 Q.mu = M.mu; Q.nu = N.nu; Q.tu = 0;
 if (M.tu*N.tu != 0) {   // Q是非零矩阵
   for (arow=1; arow<=M.mu; ++arow) {
     // 处理M的每一行
   } // for arow
 } // if
 return OK;
} // MultSMatrix
```

处理M的每一行

```
ctemp[] = 0;                    // 当前行各元素累加器清零
    Q.rpos[arow] = Q.tu+1;
for (p=M.rpos[arow]; p<M.rpos[arow+1];++p) {
 //对当前行中每一个非零元
  brow=M.data[p].j; // 左矩阵列号选定右矩阵的行
  if (brow < N.mu )  t = N.rpos[brow+1];
     else  { t = N.tu+1 }
     for (q=N.rpos[brow];  q< t;  ++q) {
       ccol = N.data[q].j;          // 乘积元素在Q中列号
       ctemp[ccol] += M.data[p].e * N.data[q].e;
     } // for q
   } // 求得Q中第crow( =arow)行的非零元
   for (ccol=1; ccol<=Q.nu; ++ccol) if (ctemp[ccol])
     if (++Q.tu > MAXSIZE) return ERROR;
     Q.data[Q.tu] = {arow, ccol, ctemp[ccol]};
   } // if
```

分析上述算法的时间复杂度

累加器ctemp初始化的时间复杂度为O(M.mu×N.nu)，

求Q的所有非零元的时间复杂度为O(M.tu×N.tu/N.mu)，

进行压缩存储的时间复杂度为O(M.mu×N.nu)，

总的时间复杂度就是O(M.mu×N.nu+M.tu×N.tu/N.mu)

若M是m行n列的稀疏矩阵，N是n行p列的稀疏矩阵，

则M中非零元的个数 $M.tu = \delta_M \times m \times n$，

N中非零元的个数 $N.tu = \delta_N \times n \times p$，

相乘算法的时间复杂度就是 $O(m \times p \times (1+n\delta_M\delta_N))$ ，

当$\delta_M<0.05$ 和$\delta_N<0.05$及 $n<1000$时，

相乘算法的时间复杂度就相当于 $O(m \times p)$。

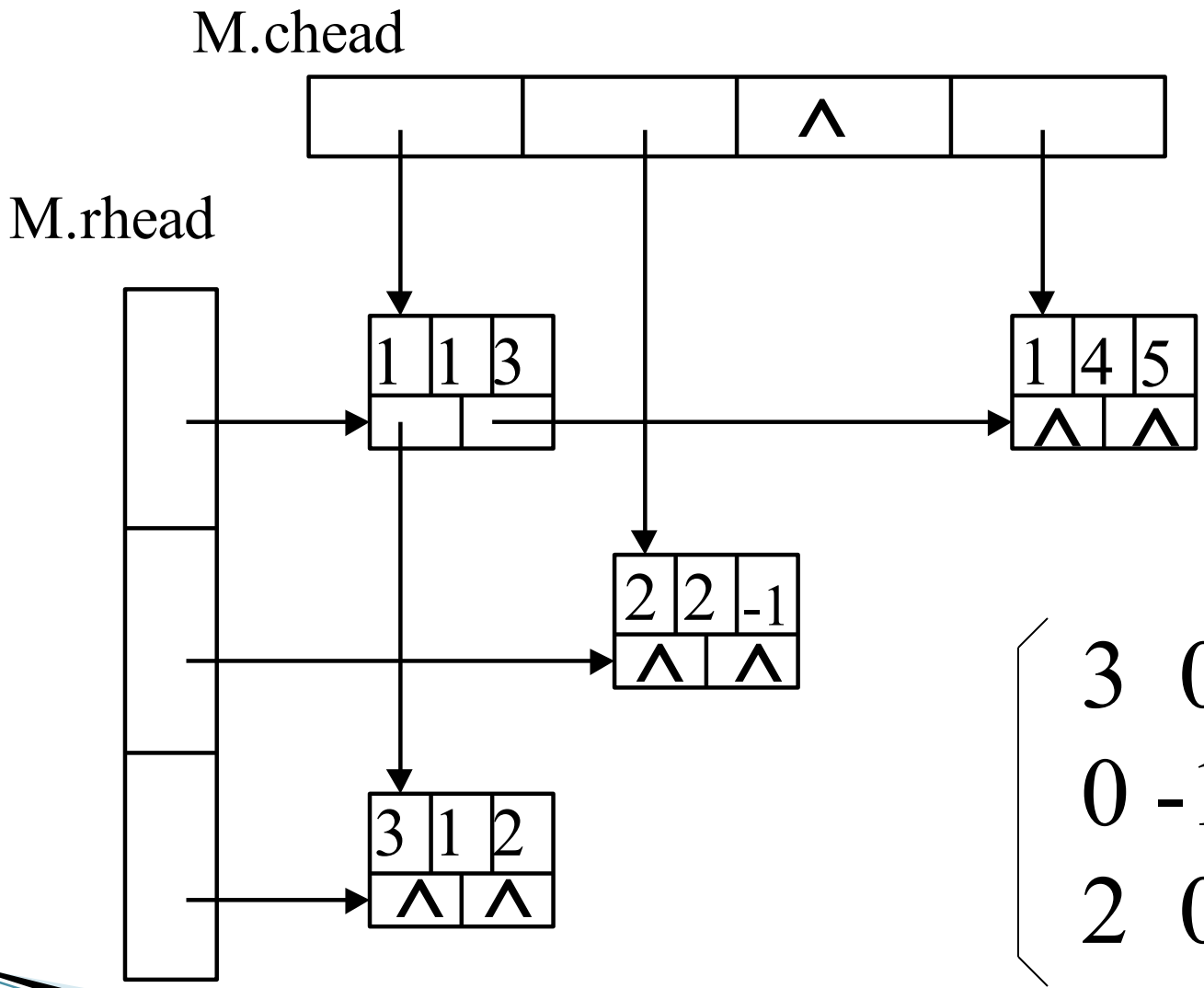
三、十字链表

- 三元组顺序表的局限：若非零元个数和位置在操作中变化较大，则会造成大量的移动——考虑链式存储。
- ·结点：非零元的信息（三元组）、所在行和列的后继链域
- ·“十字”：每一个非零元结点既是某行链表中的一个结点，又是某列链表中的一个结点。
- ·稀疏矩阵的规模：行数、列数以及非零元的个数

十字链表类型定义

```
typedef        struct  OLNode{
    int                i, j;
    ElemType           e; struct OLNode
  *right, *down; /* 非零元所在行表和列表的后继链域*/
}OLNode, *OLink;
  typedef      struct{
 OLink         *rhead, *chead;
  /* 行和列链表头指针   */
int     mu, nu, tu;              /* 矩阵的行数、列数和非零元个数 *
}CrossList;
```

三、十字链表



$$\begin{pmatrix} 3 & 0 & 0 & 5 \\ 0 & -1 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{pmatrix}$$

十字链表插入非零元

定位行链并检查重复

行列从 1 编号。先初始化 rhead 和 chead。行链按列号、列链按行号递增。成功返回 1，失败返回 0。

```
#include <stdlib.h>

typedef struct OLNode {
    int i, j, e;
    struct OLNode *right, *down;
} OLNode, *OLink;

typedef struct {
    OLink *rhead, *chead;
    int mu, nu, tu;
} CrossList;

int cross_insert(CrossList *matrix, int i, int j, int e)
{
    if (i < 1 || i > matrix->mu || j < 1 || j > matrix->nu || e == 0) {
        return 0;
    }
    OLink *row = &matrix->rhead[i];
    while (*row && (*row)->j < j) {
        row = &(*row)->right;
    }
    if (*row && (*row)->j == j) {
        return 0;
    }
```

十字链表插入非零元

定位列链并接入同一结点


```
    OLink *col = &matrix->chead[j];
    while (*col && (*col)->i < i) {
        col = &(*col)->down;
    }
    OLink node = malloc(sizeof *node);
    if (!node) {
        return 0;
    }
    *node = (OLNode){i, j, e, *row, *col};
    *row = node;
    *col = node;  // 行链和列链指向同一结点
    ++matrix->tu;
    return 1;
}
```

5.4 广义表的类型定义

ADT Glist {

数据对象： $D = \{e_i \mid i=1,2,..,n;\ n\geq 0;$

$e_i \in AtomSet$ 或 $e_i \in GList,$

AtomSet为某个数据对象 $\}$

数据关系：

$$LR = \{<e_{i-1}, e_i> \mid e_{i-1}, e_i \in D, 2\leq i\leq n\}$$

基本操作:

} ADT Glist

广义表是**递归**定义的**线性结构**，

$$LS = (\alpha_1, \alpha_2, \cdots, \alpha_n)$$

其中：α_i 或为原子 或为广义表

例如： $A = ( )$

$F = (d, (e))$

$D = ((a,(b,c)), F)$

$C = (A, D, F)$

$B = (a, B) = (a, (a, (a, \cdots ,)))$



广义表是一个**多层次**的**线性结构**

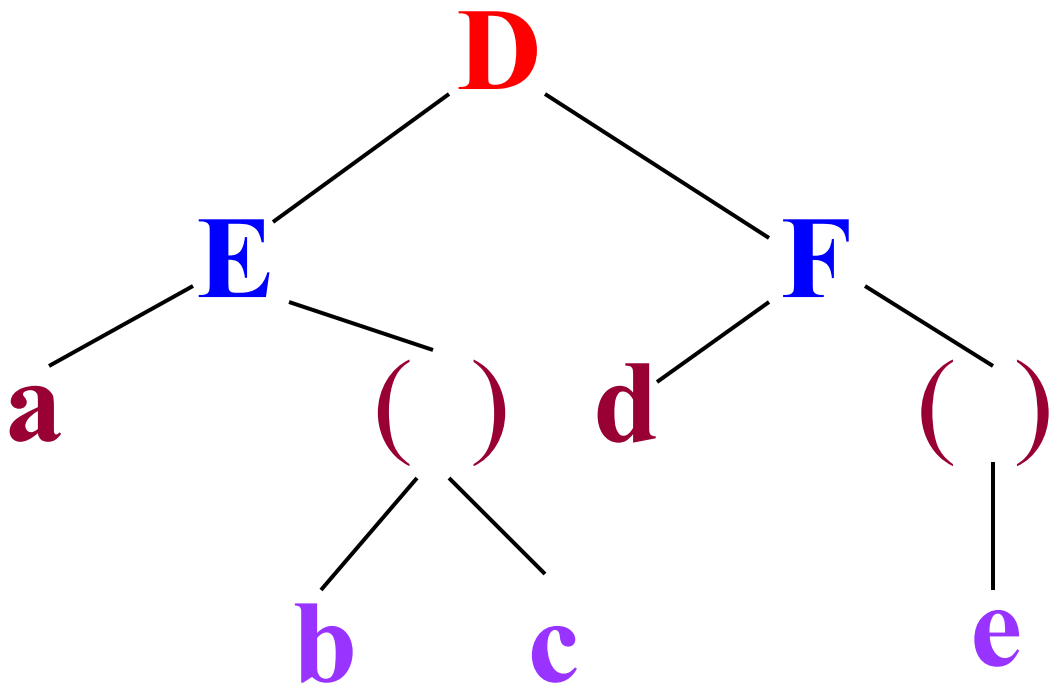
例如：

D=(E, F)

其中:

E=(a, (b, c))

F=(d, (e))



广义表 LS = ($\alpha_1, \alpha_2, \ldots, \alpha_n$)的结构特点:

1) 广义表中的数据元素有相对**次序**;

2) 广义表的**长度**定义为最外层包含元素个数;

3) 广义表的**深度**定义为所含括弧的重数;

注意: “原子”的深度为 0

“空表”的深度为 1

4) 广义表可以**共享**;

5) 广义表可以是一个**递归**的表。

递归表的深度是无穷值，长度是有限值。

6) 任何一个**非空广义表** LS = (α1, α2, …, αn)
均可分解为

表头 Head(LS) = α1 和

表尾 Tail(LS) = (α2, …, αn) 两部分。

例如: D = (E, F) = ((a, (b, c)), F)

Head(**D**) = E Tail(**D**) = (F)

Head(**E**) = a Tail(**E**) = ((b, c))

Head(**((b, c))**) = (b, c) Tail(**((b, c))**) = ( )

Head(**(b, c)**) = b Tail(**(b, c)**) = (c)

Head(**(c)**) = c Tail(**(c)**) = ( )



基本操作

- **结构的创建和销毁**

  InitGList(&L);　　DestroyGList(&L);

  CreateGList(&L, S);　CopyGList(&T, L);

- **状态函数**

  GListLength(L);　GListDepth(L);

  GListEmpty(L);　GetHead(L);　GetTail(L);

- **插入和删除操作**

  InsertFirst_GL(&L, e);

  DeleteFirst_GL(&L, &e);

- **遍历**

  Traverse_GL(L, Visit());



5.5 广义表的表示方法

通常采用头、尾指针的链表结构

表结点:

tag=1	hp	tp

原子结点:

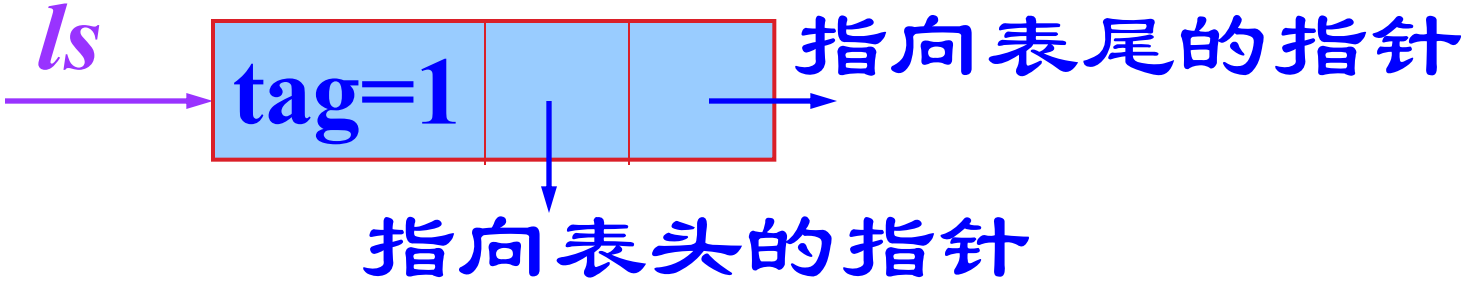
tag=0	data

构造存储结构的两种分析方法:

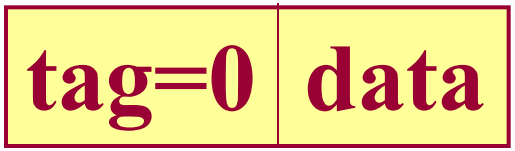
1) 表头、表尾分析法:

空表　　*ls=NIL*

非空表 *ls*

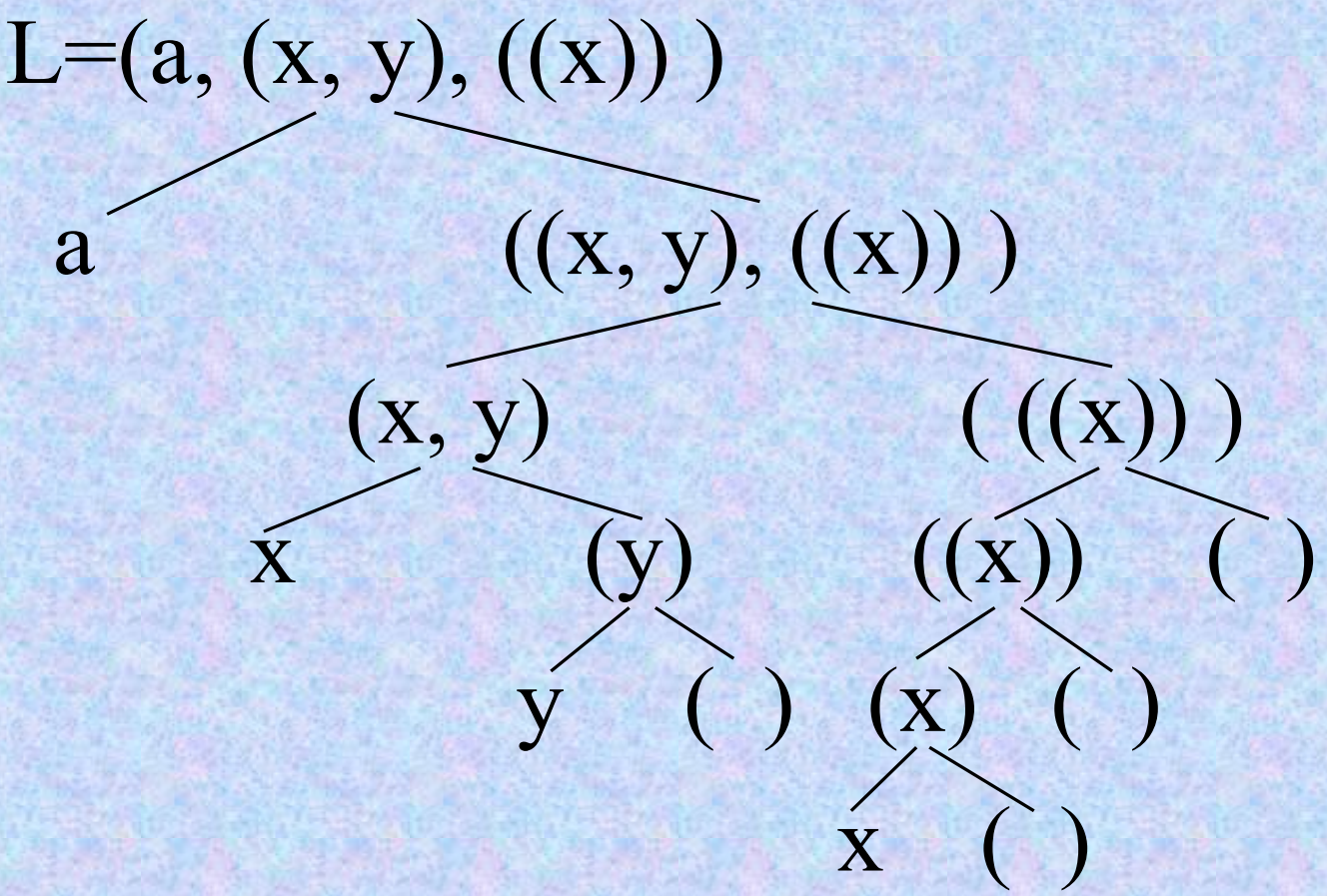


若表头为原子，则为

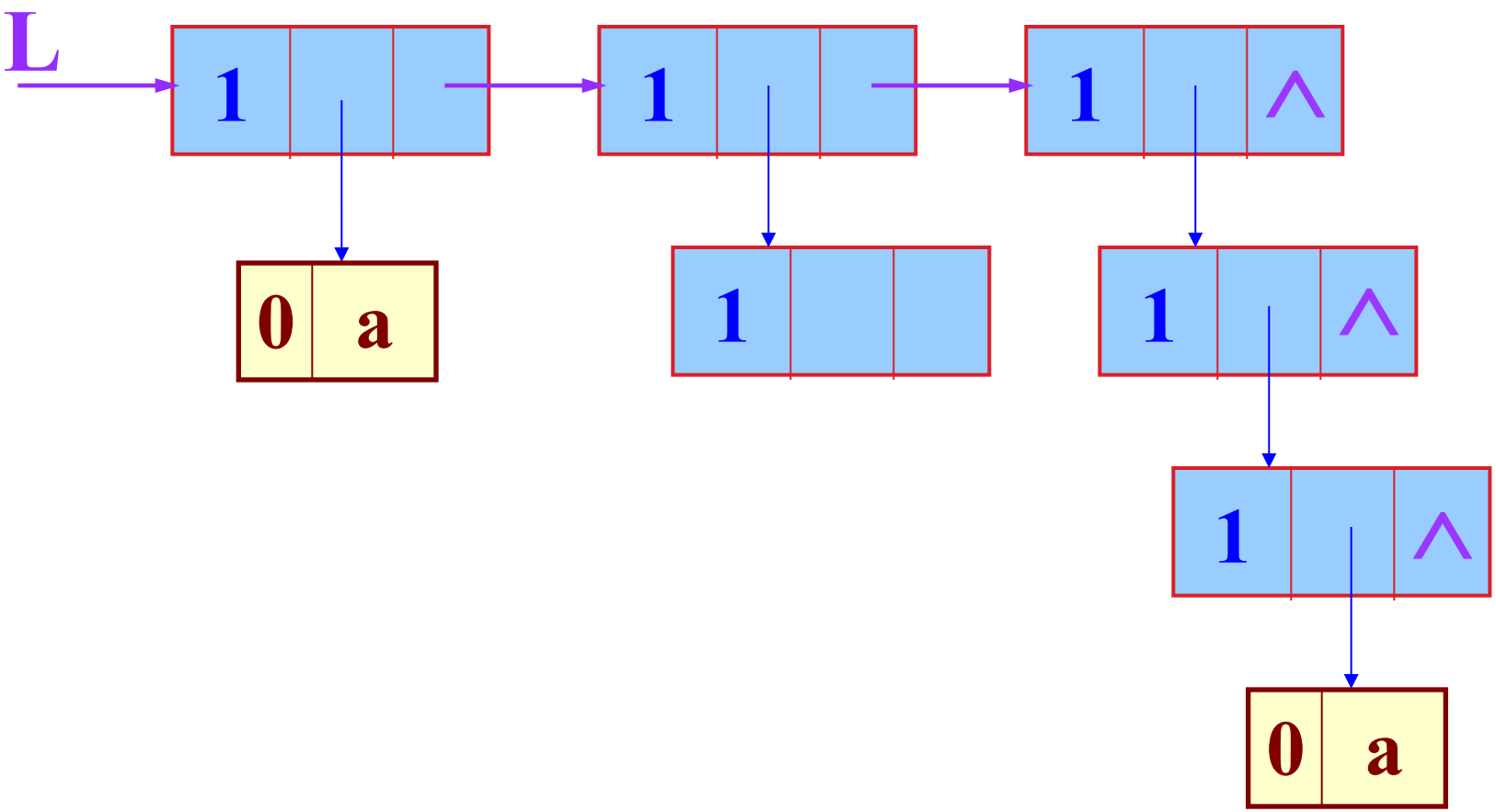


否则，依次类推。

例如：



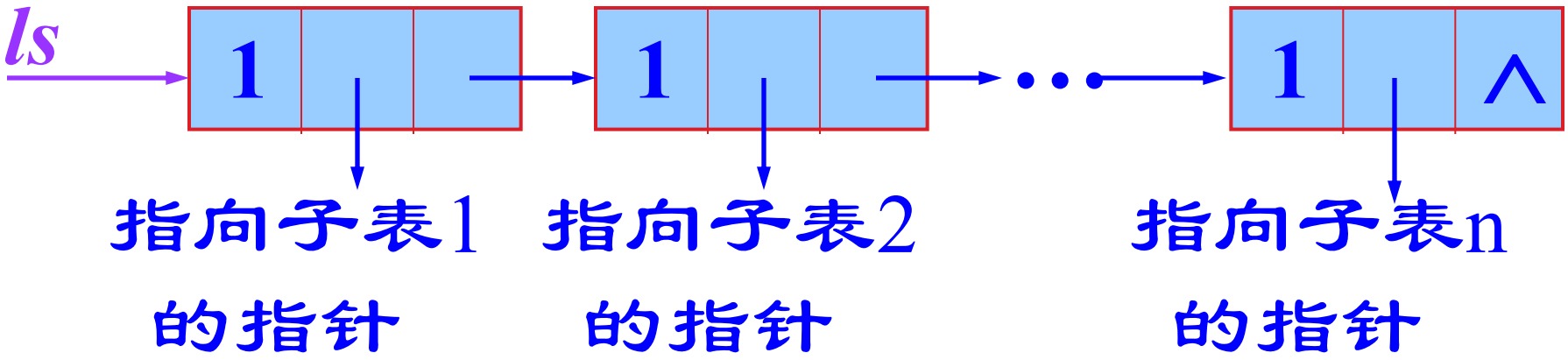
L = (a, (x, y), ((x)))



2) 子表分析法：

空表 $ls=NIL$

非空表

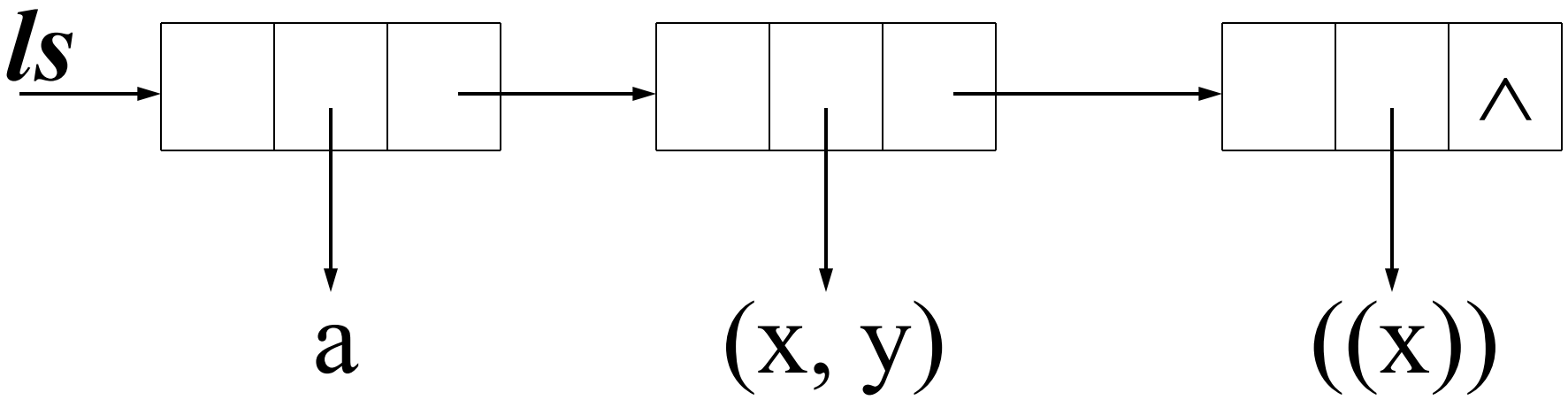


若子表为原子，则为

tag=0	data

否则，依次类推。

例如： LS=(a, (x,y), ((x)))



广义表从结构上可以分解成

广义表 = 表头 + 表尾

或者

广义表 =

子表1 + 子表2 + … + 子表n

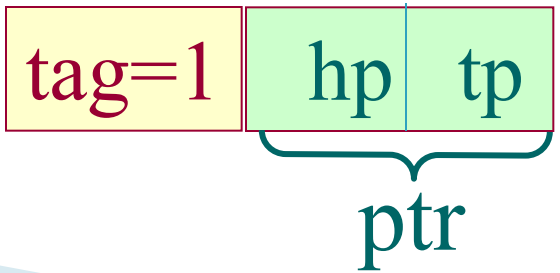
因此常利用分治法求解之。

算法设计中的关键问题是，**如何将 l 个子问题的解组合成原问题的解。**

广义表的头尾链表存储表示：

```
typedef enum {ATOM, LIST} ElemTag;
        // ATOM==0:原子, LIST==1:子表
typedef struct GLNode {
   ElemTag  tag;   // 标志域
   union{
     AtomType  atom;      // 原子结点的数据域
     struct {struct GLNode *hp, *tp;} ptr;
   };
} *GList
```

表结点



1．了解数组的两种存储表示方法，并掌握数组在以行为主的存储结构中的地址计算方法。
2．掌握对特殊矩阵进行压缩存储时的下标变换公式。
3．了解稀疏矩阵的两类压缩存储方法的特点和适用范围，领会以三元组表示稀疏矩阵时进行矩阵运算采用的处理方法。

4. 掌握广义表的结构特点及其存储表示方法，读者可根据自己的习惯熟练掌握任意一种结构的链表，学会对非空广义表进行分解的两种分析方法：即可将一个非空广义表分解为表头和表尾两部分或者分解为n个子表。

十字链表插入非零元

同时插入行链和列链

输入 3 行 3 列矩阵已有 (1,1,3)、(2,3,5)，插入 (1,3,7)。三元组为（行，列，值）。

预期 返回 1，非零元数为 3。第 1 行的列号为 1、3。第 3 列的行号为 1、2。

重复位置

输入 在上述结果中再插入 (1,3,9)。

预期 返回 0，位置 (1,3) 的值仍为 7，非零元数仍为 3。